Казанский федеральный университет
Институт информационных технологий
и интеллектуальных систем

Программная инженерия

Отчёт по блоку заданий 1

Развёртывание моделей машинного обучения
API и автоматическое тестирование
Рекомендательные системы

Выполнил
Баширов Адель Тагирович
Группа 11-411

Казань 2026

1 Цель и состав работы

Цель работы — развернуть готовые модели для разных типов данных, предоставить одну из моделей через HTTP API и сравнить алгоритмы рекомендаций на открытом наборе оценок фильмов. Результат включает исполняемый код, данные для воспроизведения эксперимента с изображениями, метрики, автоматические тесты и документацию.

Для изображений получен macro-F1 0,8436 на 45 скриншотах; API проверен настоящим HTTP-запросом. В рекомендациях item-based collaborative filtering достиг NDCG@10 0,1651 против 0,1073 у популярности. Дополнительно запущена локальная языковая модель Qwen.

Компонент	Решение	Артефакты
Текст	RuBERT — тональность отзывов	text/
Аудио	YAMNet — классы звуков ESC-10	audio/
Изображения	CLIP — тип веб-страницы	image/
Видео	YOLO11 — детекция объектов	video/
LLM	Qwen2.5-0.5B-Instruct	llm/
API	FastAPI, PyTest, GitHub Actions	api/, tests/
Рекомендации	Пять сравниваемых алгоритмов	recsys/

Происхождение результатов

Эксперименты с текстом, аудио и видео были выполнены ранее; их исходные JSON и CSV сохранены в репозитории. В текущей проверке повторно выполнены отдельные предсказания всех трёх моделей. Полные прежние оценки не пересчитывались. Сбор скриншотов, эксперименты Images и RecSys, запуск LLM и проверка API выполнены 12 октября 2026 года.

Основное окружение — Python 3.11 на macOS с Apple MPS. Для TensorFlow и Ultralytics используются отдельные окружения. Файл verification/environment-macos.txt фиксирует версии основного окружения. Большие веса моделей и исходные MovieLens, ESC-10 и COCO загружаются отдельно.

Репозиторий

https://github.com/AdelBashiroff/ProgIngenering-s1sz1

Каждый модуль имеет README с постановкой задачи, командами запуска, результатами и ограничениями. Общий отчёт объединяет результаты всех компонентов, а подробные таблицы предсказаний остаются в репозитории.

2 Анализ тональности текста

Задача — классифицировать русскоязычный отзыв как positive, negative или neutral. Используется готовая модель seara/rubert-tiny2-russian-sentiment через Transformers. Вход ограничен 256 токенами; модель выбирает класс и возвращает score. Дообучение не проводится.

Оценка выполнена на собственном наборе из 151 размеченного отзыва. Сравниваются компактный RuBERT-tiny2 и RuBERT-base. Основная метрика macro-F1 одинаково учитывает все три класса независимо от их частоты.

Модель	Accuracy	Macro-F1
RuBERT-tiny2	0,7219	0,7291
RuBERT-base	0,6689	0,6696

Для tiny2 95% доверительный интервал accuracy составляет [0,6490; 0,7949], macro-F1 — [0,6566; 0,8023]. В парном сравнении интервал разности macro-F1 tiny2 и base включает ноль: статистически убедительное превосходство на этом наборе не установлено. Компактная модель выбрана как практичный вариант для ноутбука.

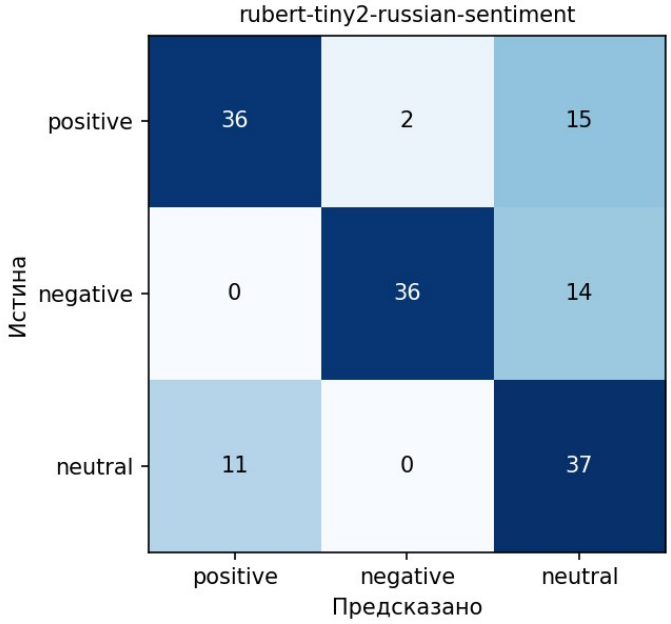


Рисунок 1 — Матрица ошибок RuBERT-tiny2 на 151 отзыве

Наиболее сложны смешанные, нейтральные и контекстно зависимые отзывы. Score модели не гарантирует правильный ответ; небольшой вручную собранный набор ограничивает переносимость оценки.

Повторная проверка: «Отличный сервис, всё понравилось!» → positive, score 0,9964. Это проверка работоспособности, а не дополнительная оценка точности.

```
python text/sentiment.py "Отличный сервис, всё понравилось!"
```

3 Классификация звуков

YAMNet из TensorFlow Hub распознаёт 521 класс AudioSet. Для учебной задачи используются десять классов ESC-10; соответствие между метками ESC-10 и AudioSet задано явно. WAV приводится к моно, float32 и частоте 16 кГц. Оценки временных фрагментов усредняются либо агрегируются максимумом.

Набор содержит 400 записей по 5 секунд, по 40 на класс. Сравниваются mean и max; модель не обучается на этих данных. В закрытой задаче предсказание выбирается только из десяти заранее известных классов.

Агрегация	Accuracy	Macro-F1
mean	0,9425	0,9426
max	0,9400	0,9401

Для mean macro-F1 равен 0,9426, 95% ДИ [0,9187; 0,9636]. Интервал accuracy — [0,9200; 0,9650]. В открытой задаче по всем 521 классам top-1 попадание в сопоставленные метки составляет лишь 0,2375: модель нередко выдаёт более общее название звука.

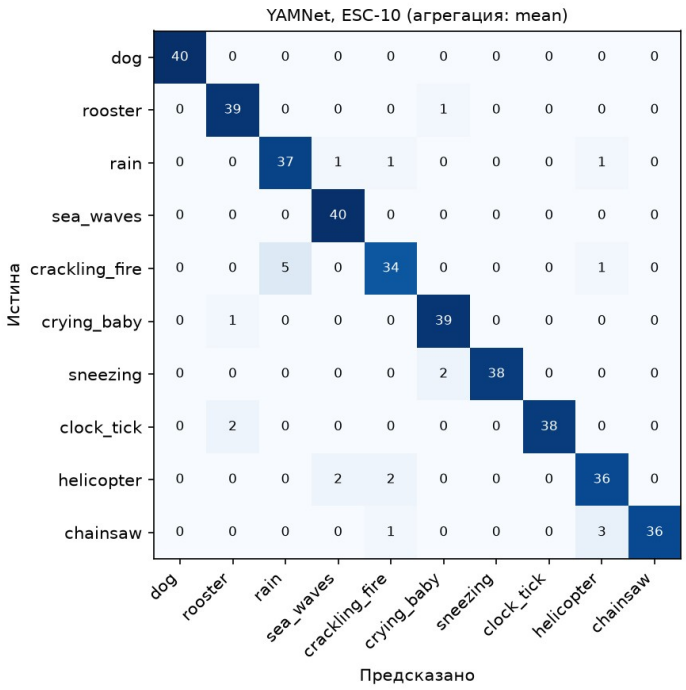


Рисунок 2 — Матрица ошибок YAMNet с агрегацией mean

Треск огня, дождь и шум волн иногда смешиваются из-за сходного шумового спектра. Это гипотеза по матрице ошибок. При повторном запуске файл 1-47273-A-21.wav распознан как sneezing, score 0,2812; модель успешно загрузилась и обработала аудио.

4 Детекция объектов на видео

Для покадровой детекции применяются готовые YOLO11n и YOLO11s из Ultralytics. Модель возвращает классы COCO, координаты рамок и confidence. Программа записывает размеченное видео, CSV детекций и JSON со скоростью и статистикой. Трекинг уникальных объектов не выполняется.

Качество оценивается на случайной выборке 500 изображений COCO val2017, seed=42. Основная метрика AP@[0,5:0,95] усредняет точность детекции по порогам IoU. Это оценка на изображениях; отдельной разметки видео для вычисления AP нет.

Модель	AP	AP50	AP small	мс/изобр.
YOLO11n	0,415	0,571	0,226	35,1
YOLO11s	0,484	0,653	0,310	47,2

YOLO11s повышает AP на 0,069, особенно для мелких объектов, но требует больше вычислений. Для демонстрационного видео использован YOLO11n. Критерии проекта AP ≥ 0,30 и не менее 5 кадров/с выполнены в сохранённом эксперименте.

Характеристика видео	Результат
Разрешение и частота	3840×2160, 59,91 кадра/с
Длительность и объём	29,48 с; 1766 кадров
Скорость полной обработки	13,0 кадров/с
Инференс модели	21,5 мс/кадр
Всего покадровых детекций	48 207
Люди / автомобили	24 204 / 23 576 детекций

Обработка медленнее реального времени для входа 60 кадров/с. Значительная доля времени уходит на чтение, отрисовку и сохранение 4K-кадров. Число детекций не равно числу уникальных людей или машин: один объект учитывается снова в каждом кадре.

Основные ограничения — мелкие и частично скрытые объекты, плотные группы и возможные ложные срабатывания редких классов. Увеличение входного разрешения или более крупная модель могут помочь ценой скорости; это не проверенные улучшения.

Повторная проверка выполнена на сохранённом изображении COCO 000000047571.jpg: найдены три объекта, в том числе horse с confidence 0,9127 и person с confidence 0,8315. Исходное демонстрационное видео хранится по ссылке из video/videos/my_video.txt; полный повторный прогон этого ролика не выполнялся.

5 Классификация скриншотов сайтов

Задача Images — определить тип страницы: landing, shop или blog. CLIP ViT-B/32 кодирует изображение и текстовые описания в общее пространство. После сравнения векторов выбирается наиболее близкий класс. Основная конфигурация pad + descriptive выбрана до оценки и используется также в API.

Собраны 73 URL. Успешно получены 56 скриншотов, 17 страниц дали ошибку загрузки. После визуальной проверки исключены ещё 11 кадров с заглушками, незагрузившимся содержимым и блокирующими окнами. Итоговый набор содержит 45 кадров: 18 лендингов, 9 магазинов и 18 блогов.

Снимки сделаны Chromium в окне 1440×900 и сохранены в JPEG. Манифест содержит URL, финальный адрес, время, HTTP-статус, SHA256 и решение QA. Изображения исключались по качеству захвата до запуска классификатора. Принятые кадры сохранены в Git для воспроизводимости.

Обработка	Промпты	Accuracy	Macro-F1
pad	ensemble	0,8667	0,8667
crop	descriptive	0,8444	0,8500
pad	descriptive	0,8444	0,8436
crop	ensemble	0,8444	0,8430
crop	simple	0,7778	0,7778
pad	simple	0,7556	0,7525

Основная конфигурация правильно классифицировала 38 из 45 страниц: accuracy 0,8444, macro-F1 0,8436. 95% ДИ macro-F1 [0,7159; 0,9363], bootstrap 1000 повторов. Порог проекта 0,70 выполнен. Лучший из шести вариантов, pad + ensemble, имеет macro-F1 0,8667; его выбор и оценка используют один набор, поэтому требуется независимое подтверждение.

Класс	Precision	Recall	F1	N
landing	0,7727	0,9444	0,8500	18
shop	0,8000	0,8889	0,8421	9
blog	1,0000	0,7222	0,8387	18

6 Ошибки и ограничения Images

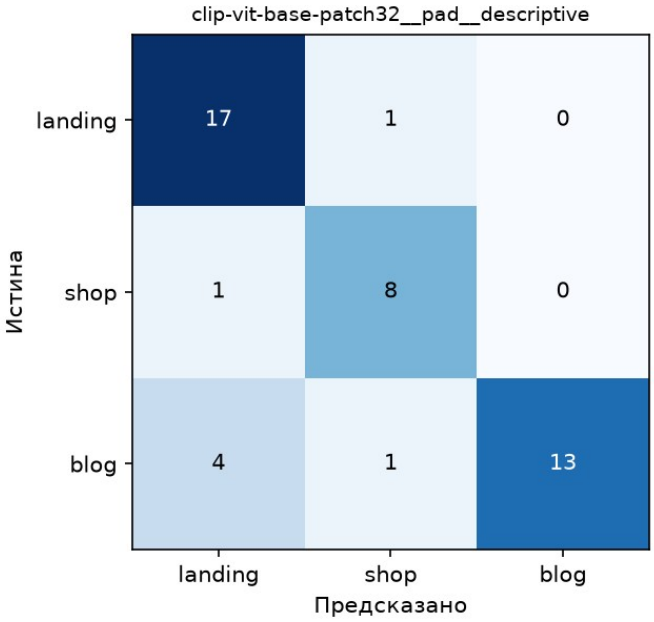


Рисунок 3 — Матрица ошибок основной конфигурации CLIP

Пять блогов приняты за лендинги либо магазин. У Stack Overflow и CSS-Tricks верх страницы содержит крупные рекламоподобные блоки; блог Ed Zitron показывает платные подписки. SparkFun определён как landing, Tilda — как shop. Это визуальные объяснения возможных причин, а не измерение внутренних механизмов CLIP.

Ошибка на Stack Overflow получила score 0,9196. Следовательно, высокий softmax не гарантирует правильности и не должен интерпретироваться как калиброванная вероятность.

Порог score	Охват	Accuracy
0,0	100,00%	84,44%
0,5	93,33%	88,10%
0,7	75,56%	97,06%
0,9	44,44%	95,00%

Отказ от ответов ниже 0,7 повышает точность на сохранённой части набора, но оставляет без ответа примерно четверть страниц. Этот порог исследован на тех же данных и не включён в API автоматически.

Набор невелик и несбалансирован. После QA осталось 44 англоязычных страницы и одна русскоязычная; сравнение языков не обосновано. Две страницы относятся к Notion. Блокировки магазинов сузили выборку, а содержание живых сайтов меняется. Мелкий текст теряется при уменьшении входа CLIP до 224×224.

7 API и автоматическое тестирование

FastAPI предоставляет CLIP через POST /predict с multipart-полем file. Ответ содержит label, score, probabilities и имя модели. GET /health проверяет работоспособность процесса, GET /model — конфигурацию. Swagger UI доступен по /docs. Загрузка модели ленивая; инференс сериализован блокировкой.



Рисунок 4 — Настоящий скриншот работающей Swagger UI

Код ответа	Условие
200	Корректное изображение и успешный инференс
400 / 415	Пустой или повреждённый файл / неподдерживаемый формат
413	Файл более 8 МиБ или изображение более 16 млн пикселей
422 / 503	Нет обязательного файла / модель недоступна

Реальный HTTP-запрос со скриншотом магазина завершился кодом 200: label=shop, score=0,9856; время 4.713 с с учётом первого обращения. Ответ сохранён в verification/api_real_request.json.

29 автоматических тестов проверяют API, сбор ссылок, временное разбиение RecSys, расчёт ranking-метрик, исключение просмотренных фильмов и холодный старт. В тестах API модель подменяется через dependency_overrides, поэтому CI не скачивает веса. Качество модели оценивается отдельно, а HTTP-интеграция проверена настоящим запросом.

GitHub Actions успешно выполнил тесты коммита 305ab54 (run 38275422269). Проверки запускаются при push, pull request и вручную. Локальный commit запускает удалённую проверку после отправки на GitHub. Для публичного сервера дополнительно нужен лимит размера всего HTTP-запроса на reverse proxy; учебный запуск привязан к localhost.

8 Постановка задачи рекомендаций

Рекомендательная система подбирает пользователю десять фильмов, которых нет в его истории оценок train. MovieLens 100K содержит 100 000 оценок 943 пользователей для 1682 фильмов. Релевантными считаются оценки 4–5. Демографические признаки не используются.

Подход	Реализация
Popular	Количество положительных оценок в train
Bayesian	Средняя оценка со сглаживанием 20 псевдооценками
Content	TF-IDF жанров и cosine с профилем понравившихся фильмов
Item-CF	Cosine по общим положительным оценкам; 40 соседей фильма
Hybrid	Среднее нормированных content и item-CF, вес 0,5

Два классических подхода и гибрид построены с NumPy, SciPy и scikit-learn. Popular и Bayesian служат простыми эвристическими baseline. Новому пользователю либо пользователю без положительной истории выдаётся popular. Все ранее оценённые фильмы, включая отрицательные оценки, исключаются.

Временное разделение и оценка

Для каждого пользователя последние 20% событий по времени оставлены в test; одинаковые временные отметки не разделяются. Получено 78 595 train-оценок и 21 405 test-оценок. Ranking-метрики усредняются по 915 пользователям с положительными test-оценками; остальные 28 в среднее не входят.

Кандидаты — весь каталог минус история train. Популярность, средние оценки и collaborative similarity используют только train. Жанры полного каталога считаются заранее доступными. 71 фильм в test отсутствует в train; на них приходится 17 положительных test-пар. Дополнительно рассчитан warm NDCG.

Разделение временное внутри каждого пользователя, но не является глобальным временным срезом: train-события одного пользователя могут быть позже test-событий другого. Для симуляции production нужен единый cutoff и учёт даты появления объекта. Параметры заданы до эксперимента и не оптимизировались по test.

Precision@10 — доля релевантных в списке; Recall@10 — доля найденных релевантных; NDCG@10 учитывает порядок с логарифмическим снижением веса; MAP@10 усредняет precision в позициях попаданий. Покрытие — доля каталога, встретившаяся в рекомендациях. Неизвестная оценка означает отсутствие наблюдаемой релевантности, а не доказанный dislike.

9 Результаты рекомендательных алгоритмов

Метод	P@10	R@10	NDCG@10	Покрытие
popular	0,0867	0,0752	0,1073	4.46%
bayesian	0,0568	0,0485	0,0601	3.57%
content	0,0328	0,0304	0,0403	28.18%
item_cf	0,1263	0,1310	0,1651	14.51%
hybrid	0,1142	0,1182	0,1497	25.68%

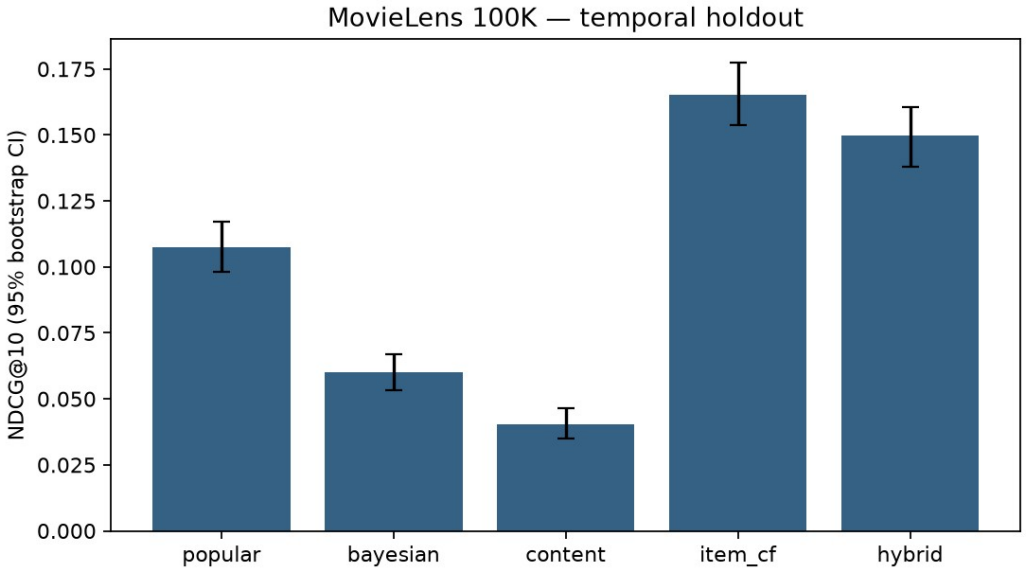


Рисунок 5 — NDCG@10 с 95% bootstrap-интервалами по пользователям

Item-CF показывает лучший NDCG@10: 0,1651 против 0,1073 у popular, относительное увеличение около 53,8%. Парная разность +0,0578 имеет 95% ДИ [0,0486; 0,0679]. Гибрид увеличивает покрытие каталога до 25,68% против 14,51% у item-CF, но уступает по NDCG. Content имеет широкое покрытие и низкую точность: одних жанров недостаточно для точного ранжирования.

Интервалы получены 1000 bootstrap-повторами по пользователям с seed=42 на фиксированном разделении. Они не учитывают перенос на иной каталог и аудиторию. Победитель выбран на том же test; для окончательного выбора алгоритма нужен отдельный контрольный период.

Ожидаемый эффект для сервиса

Гипотеза — персонализация повысит долю запусков фильмов и удержание. Offline-рост NDCG не доказывает рост CTR или выручки. Для A/B-теста предлагается рандомизация по пользователю: контроль popular против item-CF, основная метрика — доля запусков из блока рекомендаций, дополнительные — время просмотра, возврат через 7 дней и задержка p95. Размер выборки, полезный эффект и срок определяются до запуска.

10 Уязвимости и локальная языковая модель

Ограничения рекомендательной системы

Холодный старт требует fallback по популярности и стартового выбора интересов. Смещение в пользу популярных фильмов уменьшает разнообразие; следует измерять покрытие и качество по редким жанрам. Подставные оценки могут продвинуть объект, поэтому в реальном сервисе нужны лимиты, дедупликация и поиск аномалий. Эти защитные меры и атаки в эксперименте не реализованы.

MovieLens отражает оценки 1997–1998 годов и не представляет современный каталог. История оценок разрежена, а неизвестные объекты могут оказаться интересными. Для эксплуатации нужны свежие логи показов, мониторинг дрейфа и минимизация персональных данных. Плотная матрица сходств требует $O(I^2)$ памяти; при большом каталоге нужен разреженный kNN, ANN либо факторизация.

Развёртывание Qwen

Для локальной генерации используется Qwen2.5-0.5B-Instruct с 0,49 млрд параметров, лицензия Apache-2.0. Компактная модель выбрана для ноутбука. Скрипт работает через Transformers без облачного API, применяет chat template, float32 и do_sample=False. Вход ограничен 2048 токенами, генерация — максимум 512 новыми токенами.

Тестовый запрос: «Объясни в трёх предложениях, зачем нужны автоматические тесты API». Получен следующий неизменённый ответ:

1. Автоматические тесты API помогают проверить работу сервисов и систем, обеспечивая точность и стабильность.

2. Они позволяют быстро и эффективно обнаруживать ошибки или проблем с API-запросами.

3. Это также позволяет улучшать производительность и улучшать качество ответов от серверов.

Параметр запуска	Значение
Устройство	mps
Новых токенов	80
Время генерации	5.91 с
Скорость	13.54 токена/с

Модель соблюла формат трёх пунктов, но допустила грамматическую ошибку «ошибки или проблем» и неточную формулировку о повышении производительности. Тесты обнаруживают проблемы и регрессии, но сами по себе не ускоряют сервис. Пример подтверждает локальный запуск, а не общее качество LLM; сгенерированный код автоматически не исполняется.

11 Воспроизводимость и выводы

Порядок запуска

```
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt
python -m pytest -q
python -m image.evaluate
uvicorn api.main:app --host 127.0.0.1 --port 8000
python -m recsys.download_data
python -m recsys.evaluate
python -m recsys.recommender --user 1 --method item_cf
python -m llm.generate "Зачем нужны тесты API?"
```

Uvicorn запускается в отдельном терминале. Для аудио и видео применяются инструкции соответствующих README и отдельные окружения. Веса загружаются при первом запуске; затем можно использовать локальный кеш. Скриншоты Images сохранены, а MovieLens скачивается скриптом с фиксацией SHA256 архива.

Выводы

Развёрнуты модели для текста, звука, изображения и видео, а также локальная LLM. Для Images собран и проверен собственный набор; основной macro-F1 0,8436 превышает критерий 0,70. FastAPI возвращает реальные предсказания и обрабатывает ошибочные запросы. Автотесты охватывают контракт API и ключевую логику подготовки данных и рекомендаций.

В RecSys item-CF превосходит popular по NDCG на выбранном временном разделении. Гибрид улучшает покрытие ценой части точности. Перенос результатов на реальный сервис требует нового временного периода и A/B-теста. Небольшие собственные наборы, визуальные смещения и ограничения оценочного протокола явно учтены при интерпретации результатов.

Источники и материалы

1. Задания: disk.yandex.ru/i/eMj2rXfaTH0c8Q; disk.yandex.ru/i/xdvx9uKuo9cpQA; disk.yandex.ru/d/k2wM2rv8ueTwFQ.

2. CLIP: https://huggingface.co/openai/clip-vit-base-patch32

3. RuBERT: https://huggingface.co/seara/rubert-tiny2-russian-sentiment

4. YAMNet: https://tfhub.dev/google/yamnet/1; ESC-50: https://github.com/karoldvl/ESC-50

5. YOLO11: https://docs.ultralytics.com/models/yolo11/; COCO: https://cocodataset.org/

6. FastAPI testing: https://fastapi.tiangolo.com/tutorial/testing/

7. GitHub Actions: https://docs.github.com/en/actions

8. MovieLens 100K: https://grouplens.org/datasets/movielens/100k/

9. Qwen: https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct

10. Исходные метрики и предсказания: каталоги results каждого модуля в репозитории.